

R LANGUAGE COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to R

- 2. Variables and Data Types

★ 3. Vectors

- 4. Operators
- 5. Conditional Statements (if, else if, else)
- 6. Loops - for, while
- 7. Functions

★ 8. Data Frames

- 9. Lists

★ 10. Importing Data (read.csv)

★ 11. Basic Statistics Functions

- 12. Plotting Basics

★ 1. Introduction to R [IMPORTANT TOPIC]

Explanation:

R is a programming language specially designed for statistics, data analysis, and creating graphs/charts. It is one of the most popular languages among data scientists, researchers, and statisticians.

Key Points:

- Built specifically for working with data and statistics.
- Has thousands of ready-made packages for almost any kind of analysis.
- Very strong built-in support for creating charts and graphs.

Example Code:

```
print("Hello, World!")
```

Output:

```
[1] "Hello, World!"
```

 *Note: The [1] before the output in R shows the position number of the first result — this is normal R behavior.*

2. Variables and Data Types

Explanation:

In R, we usually assign a value to a variable using the special <- arrow symbol (though = also works). R automatically understands the data type from the value given.

Example Code:

```
name <- "Pravendra"  
age <- 21  
height <- 5.9  
  
print(name)  
print(age)
```

Output:

```
[1] "Pravendra"  
[1] 21
```

 *Note: <- is the traditional and most commonly used way to assign values in R.*

★ 3. Vectors *[IMPORTANT TOPIC]*

Explanation:

A vector is the most basic and important data structure in R — it stores multiple values of the SAME type together, in order. Almost everything in R is built using vectors.

Example Code:

```
numbers <- c(10, 20, 30, 40)
print(numbers)
print(numbers[1])  # R starts counting from 1, not 0!
print(sum(numbers))
```

Output:

```
[1] 10 20 30 40
[1] 10
[1] 100
```

 *Note: Unlike most programming languages, R's index starts from 1, not 0.*

4. Operators

Explanation:

Operators perform actions on values, such as maths or comparisons.

Example Code:

```
a <- 10  
b <- 3  
print(a + b)    # 13  
print(a %% b)   # 1 (remainder in R uses %%)
```

Output:

```
[1] 13  
[1] 1
```

5. Conditional Statements (if, else if, else)

Explanation:

Conditional statements let the program make decisions based on whether something is true or false.

Example Code:

```
marks <- 75

if (marks >= 90) {
  print("Grade A")
} else if (marks >= 60) {
  print("Grade B")
} else {
  print("Grade C")
}
```

Output:

```
[1] "Grade B"
```

6. Loops - for, while

Explanation:

Loops repeat a block of code multiple times, most commonly used in R to go through each item in a vector.

Example Code:

```
for (i in 1:3) {  
  print(paste("Count:", i))  
}
```

Output:

```
[1] "Count: 1"  
[1] "Count: 2"  
[1] "Count: 3"
```

 *Note: paste() is used to join text and values together into one string in R.*

7. Functions

Explanation:

A function is a reusable block of code, created using the 'function' keyword in R.

Example Code:

```
add_numbers <- function(a, b) {  
  return(a + b)  
}  
  
total <- add_numbers(5, 7)  
print(total)
```

Output:

```
[1] 12
```

★ 8. Data Frames [IMPORTANT TOPIC]

Explanation:

A data frame is R's most important data structure for working with real data — it stores data in a table format, similar to an Excel sheet, with rows and named columns. Almost all real-world data analysis in R uses data frames.

Example Code:

```
students <- data.frame(  
  name = c("Pravendra", "Aman"),  
  marks = c(90, 85)  
)  
  
print(students)
```

Output:

	name	marks
1	Pravendra	90
2	Aman	85

9. Lists

Explanation:

A list in R can hold DIFFERENT types of values together (unlike a vector, which needs all values to be the same type) — for example, text, numbers, and even another list, all in one list.

Example Code:

```
student <- list(name = "Pravendra", age = 21, passed = TRUE)
print(student$name)
print(student$age)
```

Output:

```
[1] "Pravendra"
[1] 21
```

 *Note: The \$ symbol is used to access a named item inside a list in R.*

★ 10. Importing Data (read.csv) [IMPORTANT TOPIC]

Explanation:

R is commonly used to analyze data stored in CSV files (spreadsheet-like text files). The `read.csv()` function loads this data directly into a data frame, ready for analysis.

Example Code:

```
data <- read.csv("students.csv")  
print(head(data)) # shows the first few rows
```

Output:

(Shows the first 6 rows of the students.csv file as a data frame.)

 *Note: `head()` is very useful to quickly preview a large dataset without printing all of it.*

★ 11. Basic Statistics Functions *[IMPORTANT TOPIC]*

Explanation:

R has many ready-made functions for common statistics calculations, which is one of the main reasons it is so popular for data analysis.

Key Points:

- `mean()` -> average value
- `median()` -> middle value
- `sd()` -> standard deviation
- `summary()` -> a quick statistical overview of the data

Example Code:

```
marks <- c(70, 85, 90, 60, 95)

print(mean(marks))
print(median(marks))
```

Output:

```
[1] 80
[1] 85
```

12. Plotting Basics

Explanation:

R has strong built-in support for creating graphs directly from data, which is very useful for quickly understanding patterns and trends.

Example Code:

```
marks <- c(70, 85, 90, 60, 95)
plot(marks, type = "o", col = "blue", main = "Student Marks")
```

Output:

(Shows a line-and-point graph titled 'Student Marks' in blue.)